Background and Borders

1. Background Properties

background-color

- Sets the background color of an element.

```
body {
  background-color: lightblue;
}
```

background-image

- Sets an image as the background of an element.

```
div {
  background-image: url('image.jpg');
}
```

background-repeat

- Controls if/how a background image repeats.

Value	Description
repeat	Default. Repeats both x and y.
no-repeat	No repetition.
repeat-x	Repeats horizontally only.
repeat-y	Repeats vertically only.

```
div {
  background-image: url('pattern.png');
  background-repeat: no-repeat;
}
```

background-size

- Defines the size of the background image.

Value	Description
auto	Default size.
cover	Cover entire area (may crop).
contain	Fit inside the box (no cropping).
100px 50px	Custom width and height.

```
div {
  background-size: cover;
}
```

background-position

- Sets the position of the background image.

Value	Description
left top	Top-left corner (default)
center center	Center of the element
right bottom	Bottom-right corner
10px 20px	Custom x and y position

```
div {
  background-position: center center;
}
```

background-attachment

- Sets whether the background scrolls with the page or stays fixed.

Value	Description
scroll	Default. Background scrolls.
fixed	Background stays in place.
local	Scrolls with the content.

```
body {
  background-attachment: fixed;
}

div {
  background: url('img.jpg') no-repeat center/cover fixed;
}
```

2. Border Properties

border-style

- Sets the style of the border.

Value	Description
solid	Solid line border
dashed	Dashed border
dotted	Dotted border
double	Two lines border
none	No border

```
div {
  border-style: dashed;
}
```

border-width

- Sets the thickness of the border.

```
div {
  border-width: 2px;
}
```

border-color

- Sets the color of the border.

```
div {
  border-color: red;
}
```

border-radius

- Rounds the corners of the element.

```
div {
  border-radius: 10px;
}
```

- You can also set individual corners:

```
border-top-left-radius: 10px;
```

```
div {
  border: 2px solid blue;
}
```

3. Gradient Backgrounds

Gradients are smooth transitions between two or more colors.

linear-gradient ()

- Creates a gradient in a straight-line direction.

```
div {
  background: linear-gradient (to right, red, yellow);
}
```

Directions:

- to right, to left, to top, to bottom
- 45deg (angle)

radial-gradient ()

- Circular gradient radiating from the center or a specified point.

```
div {
  background: radial-gradient (circle, red, yellow, green);
}
```

Multi-Color Gradients

```
div {
  background: linear-gradient (to bottom, red, orange, yellow, green, blue);
}
```

Gradient with Transparency

```
div {
  background: linear-gradient (to right, rgba (255,0,0,0), rgba (255,0,0,1));
}
```

1. display Property

The display property defines how an HTML element is displayed in the document flow.

Common Values:

Value	Description
block	The element takes up the full width available. Starts on a new line. Examples: , <h1>
inline	Takes up only as much width as needed. No line break. Cannot set width/height. Examples: , <a>
inline-block	Like inline, but allows setting width, height, margin, padding.
none	Completely removes the element from the layout (invisible and no space occupied).

Example:

```
div {
  display: block;
}
span {
  display: inline;
}
button {
  display: inline-block;
}
```

2. position Property

Defines how an element is positioned in the document.

Values:

Value	Description
static	Default value. Normal flow of the page. Top, left, etc. don't work.
relative	Positioned relative to its normal position. Can use top, left, etc.
absolute	Positioned relative to the nearest positioned (non-static) ancestor or <html>.
fixed	Positioned relative to the **viewport**. Doesn't move on scroll.
sticky	Scrolls with the page until a certain position is reached, then sticks.

Example:

```
.relative-box {
  position: relative;
  top: 20px;
  left: 30px;
}
.fixed-menu {
  position: fixed;
```

```
  top: 0;
}
```

3. z-index Property

Controls the **stacking order** of elements (which element appears on top of which).

- Higher z-index = element appears on top.
- Only works on positioned elements (relative, absolute, fixed, or sticky).

Example:

```
div {
  position: absolute;
  z-index: 10;
}
```

4. Visibility vs Display

Property	Visibility	Display
visibility: hidden	Hides the element **but space remains**	
display: none	Hides the element **and removes its space**	

Example:

```
.hidden-box {
  visibility: hidden;
```

```
}

.none-box {
  display: none;
}
```

5. Overflow Property

Controls what happens if content overflows an element's box.

Values:

Value	Description
visible	Default. Content overflows visibly.
hidden	Hides the overflowing content.
scroll	Adds scrollbars even if not needed.
auto	Adds scrollbars only when necessary.

Example:

```
.box {
  width: 200px;
  height: 100px;
  overflow: auto;
```

}

Summary Table

Property	Key Use
display	How elements appear in layout
position	Placement of elements
z-index	Layering of elements
visibility	Hide/show with space
overflow	Content overflow handling

What is Flexbox?

Flexbox (Flexible Box Layout) is a one-dimensional layout system used for arranging elements in **rows or columns**. It makes it easier to align items, distribute space, and handle responsive designs.

- Efficient for creating **layouts that adjust to screen size**
- Align items **horizontally or vertically**
- Avoids using float and position for alignment

Flex Container Properties

To use Flexbox, set the container's display property to flex or inline-flex.

- **display: flex**
  - Turns an element into a **flex container**.
  - Its direct children become **flex items**.

```
.container {
  display: flex;
}
```

flex-direction

Defines the **main axis direction** (row or column layout).

Value	Description
row (default)	Left to right
row-reverse	Right to left
column	Top to bottom
column-reverse	Bottom to top

```
.container {
  flex-direction: row;
}
```

justify-content

Aligns items **along the main axis**.

Value	Description
flex-start	Items start from beginning
flex-end	Items align to end

Value	Description
center	Centered items
space-between	Equal spacing between
space-around	Equal spacing around
space-evenly	Equal spacing everywhere

```
.container {
  justify-content: center;
}
```

align-items

Aligns items **along the cross axis** (opposite of main axis).

Value	Description
stretch (default)	Items stretch to fill container
flex-start	Align to top or left
flex-end	Align to bottom or right
center	Centered

Value	Description
baseline	Align to text baseline

```
.container {
  align-items: center;
}
```

Flex Item Properties

flex-grow

Defines how much a flex item will **grow** relative to others when extra space is available.

```
. item {
  flex-grow: 1;
}
```

If all items have flex-grow: 1, they share space equally. If one has 2, it takes twice the space.

flex-shrink

Defines how much a flex item will **shrink** when there’s not enough space.

```
. item {
  flex-shrink: 1;
```

}

0 means don’t shrink at all.

flex-basis

Sets the **initial size** of a flex item **before** space is distributed.

```
. item {
  flex-basis: 150px;
}
```

Overrides width or height depending on the direction.

align-self

Overrides align-items **for individual item**.

```
. item {
  align-self: flex-end;
}
```

Useful to position one item differently than others.

Flex Wrap and Gap

flex-wrap

Allows items to wrap to the next line if needed.

Value	Description
nowrap (default)	All items on one line
wrap	Wrap to the next line
wrap-reverse	Wrap in reverse order

```
.container {
  flex-wrap: wrap;
}
```

gap

Adds spacing between flex items (horizontal and vertical).

```
.container {
  gap: 20px;
}
```

Replaces the need for margin hacks between items.

Example:

```
<style>
.container {
  display: flex;
  flex-direction: row;
```

```
  justify-content: space-around;
  align-items: center;
  flex-wrap: wrap;
  gap: 20px;
}

. item {
  flex: 1;
  flex-basis: 150px;
  background-color: lightblue;
  padding: 20px;
  text-align: center;
}
</style>

<div class="container">
  <div class="item">Box 1</div>
  <div class="item">Box 2</div>
  <div class="item">Box 3</div>
</div>
```

Introduction to Grid Layout

CSS Grid Layout is a **2-dimensional layout system**, which means it can handle both **rows and columns** simultaneously. It's ideal for building responsive, organized, and powerful layouts without relying on floats or positioning.

Grid Container

To create a grid, the parent container must use display: grid or inline-grid.

display: grid

```
.container {
  display: grid;
}
```

- The container becomes a **grid container**
- Its direct children become **grid items**

grid-template-columns and grid-template-rows

Used to define the number and size of columns and rows in the grid.

```
.container {
  display: grid;
  grid-template-columns: 200px 1fr 2fr;
  grid-template-rows: 100px auto;
}
```

Units:

- px, em, % — fixed units
- fr — **fractional unit**, distributes remaining space

Example:

```
grid-template-columns: 1fr 1fr 1fr;  /* 3 equal columns */
grid-template-rows: 100px 200px;   /* 2 rows */
```

Grid Gap

Defines the spacing between rows and columns.

```
.container {
  grid-gap: 20px; /* gap for both rows and columns */
  row-gap: 10px;
  column-gap: 30px;
}
```

Row/Column Start and End

You can place items at specific grid positions using **line numbers**.

Example:

```
.item1 {
  grid-column-start: 1;
  grid-column-end: 3;   /* Spans 2 columns */
  grid-row-start: 1;
  grid-row-end: 2;
```

```
}

.item1 {
  grid-column: 1 / 3;
  grid-row: 1 / 2;
}
```

Grid Area and Named Grid Lines

grid-area

You can assign a name to an item and position it using grid-template-areas.

Example:

```
.container {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar content"
    "footer footer";
  grid-template-columns: 200px 1fr;
  grid-template-rows: auto 1fr auto;
}

.header {
  grid-area: header;
```

```
}
.sidebar {
  grid-area: sidebar;
}
.content {
  grid-area: content;
}
.footer {
  grid-area: footer;
}
```

Named Grid Lines

You can name grid lines for more readable placement.

```
.container {
  display: grid;
  grid-template-columns: [start] 1fr [middle] 2fr [end];
}
```

Then place an item:

```
.item {
  grid-column: start / middle;
}
```

Sample Grid Code

```
<style>
.container {
```

```
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: 100px 200px;
  gap: 10px;
}

.item1 {
  grid-column: 1 / 3;
}
.item2 {
  grid-row: 2 / 3;
}
</style>

<div class="container">
  <div class="item1">Box 1 (spans 2 columns)</div>
  <div class="item2">Box 2 (on 2nd row)</div>
  <div class="item3">Box 3</div>
</div>
```

Summary Table

Property	Purpose
display: grid	Defines a grid container
grid-template-columns	Set number/size of columns
grid-template-rows	Set number/size of rows
gap, row-gap, column-gap	Control spacing
grid-column, grid-row	Place items by line numbers
grid-area	Name areas and place them
grid-template-areas	Define layout structure visually